Common Core: Phase 1 RTL Design

Lab R2: Synchronous FIFO

Parameterized RTL and Simulation Semantics

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Design a parameterized synchronous FIFO that works for any depth, and learn how a SystemVerilog simulator schedules events, why races happen, and why testbenches drive with nonblocking assignments.

Duration	1 week (approximately 15–20 hours)
Prerequisites	Labs L0 and R1
Designs	`sync_fifo` (new)
Reused in	V1 (verification), P1 and P2 (synthesis, timing)

Contents

1 Overview **3**

2 Design Specification **3**

2.1 Interface . . . 3

2.2 Behavior . . . 4

2.3 Micro-architecture . . . 4

2.4 Design Note: Pointer Wrap for Any `DEPTH` . . . 4

3 Parameterization **5**

3.1 `parameter`, `localparam` and `$clog2` . . . 5

3.2 Overriding Parameters and the Testbench Top . . . 6

3.3 Required Parameter Sets . . . 6

4 Simulation Semantics: Scheduling and Races **6**

4.1 Time Steps and Scheduling Regions . . . 6

4.2 A Rising Clock Edge, Step by Step . . . 7

4.3 What a Race Is . . . 7

4.4 The Testbench–DUT Race and Why the Testbench Uses `<=` . . . 8

4.5 Why `#0` Is Not a Fix . . . 8

4.6 Exercise: Race Demonstration . . . 8

5 Smoke Testbench **9**
5.1 Structure . . . 10
5.2 Required Tests . . . 10
5.3 Skeleton . . . 11

6 Requirements and Tasks **12**

7 Repository and Flow **13**
7.1 Files created in this lab . . . 13
7.2 File lists and Makefile . . . 13
7.3 Commands . . . 14

8 Deliverables **14**

9 Assessment **14**

10 Review Questions **15**

11 References **15**

1 Overview

FIFOs are everywhere on a chip: between a bus interface and the core, between pipeline stages that run at different rates, in front of every memory controller. A FIFO is also the simplest block whose correctness depends on *parameters*: the same RTL must work at depth 2 and at depth 1000, at width 1 and at width 128. Most real IPs are configurable in the same way, and a configuration nobody tested is a configuration that does not work.

You will design `sync_fifo`, a first-word fall-through FIFO with parameters `WIDTH` and `DEPTH`, which must work for every `DEPTH` ≥ 2 — not only powers of two. Alongside, you will learn how a simulator orders the events of one clock edge. That knowledge explains Rule 3 of Lab R1, explains why your smoke testbench in Lab R1 drove at the falling edge, and sets the testbench driving standard used in every later lab: **drive with `<=` at the rising edge**.

Where this lab sits. Lab R2 uses the flow of Lab L0 and the coding rules of Lab R1. Lab V1 verifies `sync_fifo` with random stimulus, a regression over parameter sets and assertions; Labs P1 and P2 synthesize it for two parameter sets and show how `WIDTH` and `DEPTH` change area and critical path. The pointer and full/empty ideas of this lab come back, with Gray codes and two clocks, in the asynchronous FIFO of Lab T-RTL1.

By the end of this lab you should be able to:

1. Design a parameterized FIFO with registered status flags and explain how its pointers wrap for any depth.
2. Use `parameter`, `localparam` and `$clog2` correctly, override parameters with `-G` through `PARAMS`, and pass them from a testbench top to the DUT.
3. Describe the SystemVerilog scheduling regions and predict in which order statements execute within one time step.
4. Explain what a simulation race is and recognize one in code.
5. Explain why nonblocking assignments in the testbench prevent the most common testbench–DUT race, and why `#0` does not.
6. Write a directed smoke testbench that drives with `<=` and checks the DUT against a queue model.

2 Design Specification

Copy this section into `hw/ip/sync_fifo/doc/spec.md` and keep the two consistent.

2.1 Interface

Port	Direction	Width	Description
`clk`	input	1	Clock, rising edge
`rst_n`	input	1	Asynchronous reset, active low
`push`	input	1	Write `push_data` into the FIFO
`push_data`	input	`WIDTH`	Data to write
`full`	output	1	No free entry
`pop`	input	1	Remove the head entry
`pop_data`	output	`WIDTH`	Head entry, valid whenever `!empty`
`empty`	output	1	No stored entry
`count`	output	`$clog2(DEPTH+1)`	Number of stored entries

Parameters: `WIDTH` (default 8) and `DEPTH` (default 16). The design must work for any `DEPTH` ≥ 2, not only powers of two.

2.2 Behavior

1. `pop_data` always shows the oldest entry (first-word fall-through, FWFT); `pop` removes it at the next rising edge.
2. A `push` while `full` is ignored, unless `pop` is also asserted in the same cycle; then both happen and `count` does not change.
3. A `pop` while `empty` is ignored. A simultaneous `push` is still accepted.
4. `full` and `empty` must come directly from registers or from `count`; they must not depend combinationally on `push` or `pop`.
5. After reset the FIFO is empty: `empty = 1`, `full = 0`, `count = 0`.

Why rule 4 matters. A consumer typically computes `pop` from `empty` ("pop when there is something and I can take it"). If `empty` depended combinationally on `pop`, the two would form a combinational loop through the consumer. Registered flags also give the consumer a full clock period to react, which matters for timing in Lab P2.

2.3 Micro-architecture

Figure 1 shows one implementation, at the level of detail expected in your `doc/design.md`.

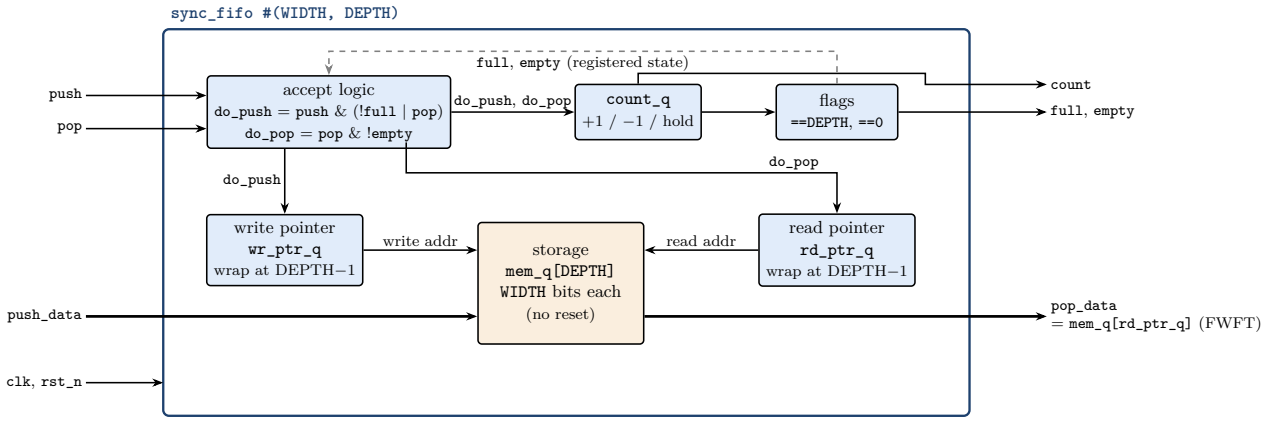


Figure 1: Block diagram of a `sync_fifo` implementation (blue: control registers and logic, all reset; orange: storage, not reset). `pop_data` is a read of the storage at the read pointer, which gives first-word fall-through for free; `full` and `empty` are compares of the `count` register, so they never depend combinationally on `push` or `pop`.

A common implementation keeps a write pointer, a read pointer and a counter, all reset, and a storage array that is not reset (Rule 6 of Lab R1: datapath only, in its own `always_ff`). `pop_data` is a read of `mem_q[rd_ptr_q]`, which gives FWFT behavior for free. Other correct solutions exist (for example, deriving `full` and `empty` from registered flags instead of the counter); any of them is acceptable if it meets the specification and you can explain it. All coding rules of Lab R1 apply.

2.4 Design Note: Pointer Wrap for Any `DEPTH`

Write a short design note in `doc/design.md` that explains how the read and write pointers wrap when `DEPTH` is not a power of two. The points it must address:

- A pointer has `$clog2(DEPTH)` bits. For `DEPTH = 16` that is 4 bits, and `ptr + 1` wraps from 15 to 0 by itself. For `DEPTH = 5` the pointer has 3 bits and would count 5, 6, 7 — addresses that do not exist.
- The wrap must therefore be explicit, for example `ptr_d = (ptr_q == PtrW'(DEPTH-1)) ? '0 : ptr_q + 1'b1;`. Show the comparison width is right (Rule 12 of Lab R1).
- Why `count` needs `$clog2(DEPTH+1)` bits and not `$clog2(DEPTH)`: it must represent both 0 and `DEPTH`.
- The "extra pointer bit" technique (full when the pointers are equal and the extra bits differ) only works for powers of two. Explain why, and why your design does not need it. (Lab T-RTL1 uses it, with Gray-coded pointers, in the asynchronous FIFO.)
- A cycle-by-cycle table for `DEPTH = 5` showing `wr_ptr`, `rd_ptr`, `count`, `full` and `empty` while the FIFO is filled, drained, and filled again past the wrap point.

3 Parameterization

3.1 `parameter`, `localparam` and `$clog2`

```
module sync_fifo #(
  parameter int unsigned WIDTH = 8,     // overridable by the instantiating module
  parameter int unsigned DEPTH = 16
) (
  input  logic                          clk,
  input  logic                          rst_n,
  // ...
  output logic [$clog2(DEPTH+1)-1:0] count
);
  // Derived constants: not overridable, always consistent with DEPTH
  localparam int unsigned PtrW = $clog2(DEPTH);
  localparam int unsigned CntW = $clog2(DEPTH + 1);

  logic [PtrW-1:0]  wr_ptr_q, rd_ptr_q;
  logic [WIDTH-1:0] mem_q [DEPTH];
  // ...
endmodule
```

- A `parameter` is part of the module's interface; anyone instantiating the module may change it. A `localparam` is derived inside the module and cannot be overridden, so it can never be set inconsistently with the parameters it comes from.
- `$clog2(n)` is the ceiling of $\log_2 n$, evaluated at elaboration time. It is the number of bits needed to address `n` items (indices 0 to `n-1`); it is *not* the number of bits needed to hold the value `n`, hence `$clog2(DEPTH+1)` for `count`. Note that `$clog2(1) = 0`: a 1-entry FIFO would need zero-width pointers, which is one reason this specification requires `DEPTH` ≥ 2.
- Give parameters an explicit type (`int unsigned`) so that widths and comparisons do not depend on the type of the default value.
- Check illegal values at elaboration time, so a bad configuration fails to build instead of simulating nonsense:

```
  // Elaboration-time check (IEEE 1800-2017, 20.11): not an assertion,
  // evaluated once when the design is built, also seen by synthesis.
  if (DEPTH < 2) begin : gen_depth_check
```

```
    $error("sync_fifo: DEPTH must be >= 2, got %0d", DEPTH);
  end
```

3.2 Overriding Parameters and the Testbench Top

Lab L0 introduced the difference between compile-time parameters and runtime plusargs. For this design it means: `WIDTH` and `DEPTH` change the hardware (number of flip-flops, pointer widths), so they can only be changed by building a new model. The flow turns `PARAMS="WIDTH=32 DEPTH=5"` into Verilator `-GWIDTH=32 -GDEPTH=5` overrides of the *testbench top*, in a separate build directory per parameter set.

For one override to change both the testbench and the DUT, the testbench top declares the same parameters and passes them down:

```
module tb_sync_fifo #(
  parameter int unsigned WIDTH = 8,
  parameter int unsigned DEPTH = 16
);
  sync_fifo #(.WIDTH(WIDTH), .DEPTH(DEPTH)) u_dut (.*);
  initial $display("PARAMS: WIDTH=%0d DEPTH=%0d", WIDTH, DEPTH);
  // ...
endmodule
```

Two typical mistakes: a testbench that instantiates the DUT without passing the parameters (the DUT silently keeps its defaults while the testbench believes it tests another configuration), and a testbench whose own constants (loop bounds, model size) are hard-coded instead of derived from `WIDTH` and `DEPTH`. Printing the parameters at the start of every run makes the first mistake visible in the log.

3.3 Required Parameter Sets

Set	`PARAMS`	Why
Default	`WIDTH=8 DEPTH=16`	Power of two, pointers wrap naturally
Non-power of 2	`WIDTH=32 DEPTH=5`	Explicit wrap; 3-bit pointers that must skip 5–7
Minimum	`WIDTH=1 DEPTH=2`	Smallest legal FIFO; 1-bit pointers and data

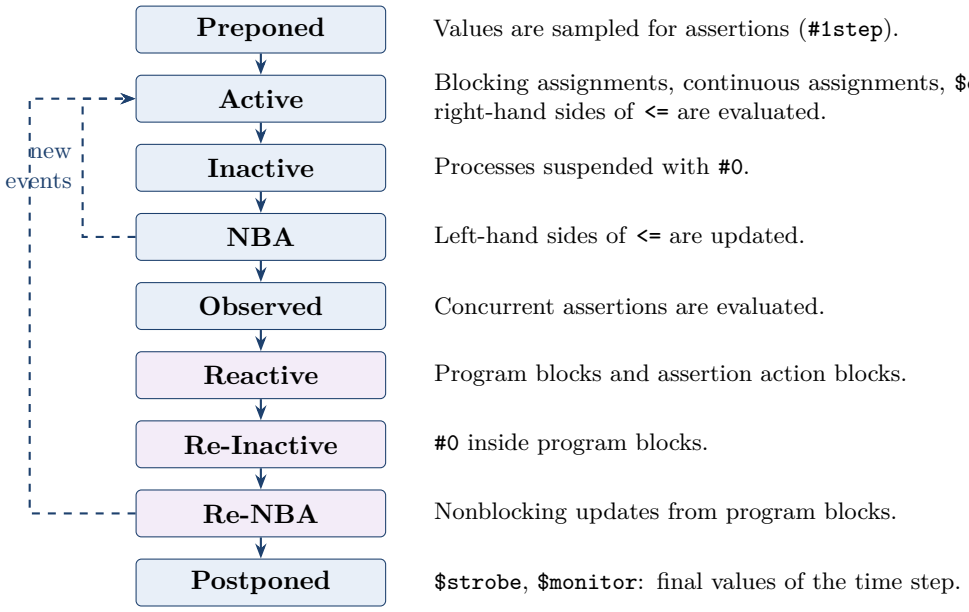
Lint and all smoke tests must pass for each set. Lab V1 adds more sets to an automated regression.

4 Simulation Semantics: Scheduling and Races

A simulator does not execute your design "in parallel". It executes processes (`always` blocks, `initial` blocks, continuous assignments) one at a time, and it decides the order using the scheduling rules in IEEE 1800, the SystemVerilog standard. When you understand those rules, you can predict exactly when a value becomes visible — and recognize code whose result the standard leaves undefined.

4.1 Time Steps and Scheduling Regions

Simulation time advances in time steps. All events that occur at the same simulation time belong to one time step. Within a time step, the simulator processes events in a fixed sequence of *regions*.



When the NBA region updates a variable, every process sensitive to that variable is triggered again in the Active region. The simulator loops through Active, Inactive and NBA until no new events remain; only then does it move on. The regions marked in purple belong to the reactive region set, used by program blocks and assertion action blocks. The Preponed and Observed regions are where concurrent assertions sample and evaluate; you will use them in Lab V1, and the reactive regions again with clocking blocks (Lab V2) and UVM (Lab T-DV2).

4.2 A Rising Clock Edge, Step by Step

Consider a flip-flop and a combinational gate:

```
always_ff @(posedge clk) q <= d;   // process P1
assign y = q & en;                 // process P2
```

At the rising edge of `clk`:

1. **Active:** `clk` changes to 1. P1 wakes up and evaluates the right-hand side `d`, using the value `d` has *now*. The update of `q` is scheduled, not performed.
2. **NBA:** `q` takes the new value.
3. **Active again:** the change of `q` triggers P2, which updates `y`.
4. **Postponed:** `$strobe` would print the final values of `q` and `y` for this time step.

Because every flip-flop evaluates its input in step 1, before any flip-flop output changes in step 2, all flip-flops sample the values from *before* the edge — exactly like real hardware. This is the reason for Rule 3 of Lab R1.

4.3 What a Race Is

A **race** occurs when two processes run in the same region of the same time step, at least one of them writes a variable and another reads or writes it, and the result depends on which process runs first. The standard does not define the order of processes within a region, so different simulators — or the same simulator after a small code change — may give different results. A race is a bug in the code, even if the simulation happens to pass today.

```
// RACE: both processes wake up in the Active region at the same edge
always @(posedge clk) a = b;   // P1 writes a
always @(posedge clk) c = a;   // P2 reads a
// P1 first -> c gets the new a.  P2 first -> c gets the old a.
```

4.4 The Testbench–DUT Race and Why the Testbench Uses <=

The most common race in verification is between the testbench driving a DUT input and the DUT sampling it on the same clock edge:

```
// DUT
always_ff @(posedge clk) q <= d;

// Testbench: BAD -- blocking drive at the active clock edge
initial begin
  @(posedge clk);
  d = 1'b1;          // Active region, same as the DUT's evaluation of d
end
// If the testbench runs first, the DUT samples the new d (1) at this edge.
// If the DUT runs first, it samples the old d (0).  -> race

// Testbench: GOOD -- nonblocking drive
initial begin
  @(posedge clk);
  d <= 1'b1;         // updated in the NBA region, after every flop has
end                  // evaluated d in the Active region
// The DUT always samples the old d at this edge and the new d at the
// next edge -- exactly as if d came from another flip-flop.
```

With <=, the testbench behaves like a flip-flop clocked by the same clock: whatever order the processes run in, the DUT sees the new value one edge later. This makes the testbench deterministic and makes the waveforms look like real synchronous hardware.

The smoke testbench of Lab R1 avoided the same race differently, by driving on the falling edge. Both methods are correct. From this lab on, the standard is: **the testbench drives DUT inputs with <= at the rising edge and samples DUT outputs at the rising edge**. A testbench process that wakes up on the rising edge reads values in the Active region, before the NBA updates of the same edge, so it sees the values the DUT sampled — the same values a real flip-flop would capture.

SystemVerilog also offers *clocking blocks*, which define exactly when a testbench samples and drives relative to the clock. They are introduced in Lab V2; understanding the regions above is the prerequisite.

4.5 Why #0 Is Not a Fix

A common "fix" for a race is a zero delay: `#0 d = 1;`. It moves the process to the Inactive region, which runs after the Active region. It only moves the race: as soon as another process also uses `#0`, the order is undefined again. Never use `#0` to solve a race.

4.6 Exercise: Race Demonstration

Write `hw/ip/sync_fifo/tb/race_demo.sv` containing the four cases below. For each case, *first write down your prediction* with a reference to the scheduling regions, then run it and compare.

1. Two `always @(posedge clk)` blocks with blocking assignments that write and read the same

variable.

2. A flip-flop driven by a testbench with a blocking assignment at the rising edge, then with a nonblocking assignment.
3. A variable updated with `<=`, printed in the same time step with `$display`, `$strobe` and `$monitor`.
4. Case 1 "fixed" with `#0` in one block, then in both blocks.

The race demo is not part of the FIFO testbench and not in any file list. Run it directly with Verilator, writing into the build directory:

```
cd $REPO_ROOT/hw/ip/sync_fifo
verilator --binary --timing -Wno-fatal tb/race_demo.sv --top-module race_demo \
  -Mdir $REPO_ROOT/build/sync_fifo/race_demo
$REPO_ROOT/build/sync_fifo/race_demo/Vrace_demo | tee \
  $REPO_ROOT/build/sync_fifo/race_demo/race_demo.log
```

On the Synopsys cloud: VCS is an event-driven simulator, so on the cloud you can compare directly: `vcs -full64 -sverilog tb/race_demo.sv -o $REPO_ROOT/build/sync_fifo/race_demo_vcs/simv` then run `simv`. Expect real differences: Verilator 5.013 warns `ZERODLY` ("`#0` delays do not schedule process resumption in the Inactive region") and ignores `#0`, and its `--timing` scheduler may let a flip-flop see a value the testbench assigned with `<=` at the same edge. Put both logs in the README. For the FIFO itself use `make sim SIM=vcs` and `make lint LINT_TOOL=spyglass` (the cloud defaults); VCS shows an unwritten storage entry as X on `pop_data`.

Verilator is not a fully event-driven simulator: it orders and optimizes processes at compile time, so the outcome of a race may differ from an event-driven simulator. If possible, also run the cases on an event-driven simulator (for example on EDA Playground) and compare. A different result between two simulators is exactly what a race looks like. Note also which cases Verilator's lint already flags (for example, `BLKSEQ` in case 1), and which it cannot.

5 Smoke Testbench

As in Lab R1, the designer's smoke testbench `tb/tb_sync_fifo.sv` is small, directed and self-checking. It is the first testbench in the program written to the `<=` driving standard.

Test term — Smoke test. A short directed test of the most basic behavior — here a few pushes and pops — that answers "is the block alive and is the flow working?". A pass proves very little; a failure means nothing else is worth running. `smoke` is the default test and the one `make check-all` runs.

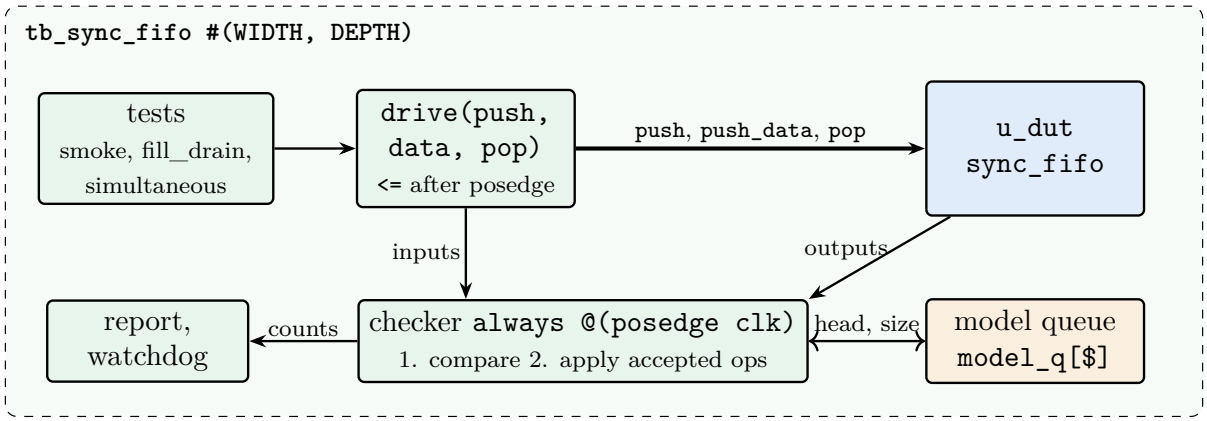


Figure 2: Structure of the R2 smoke testbench. The stimulus drives with `<=` right after a rising edge; at every rising edge the checker first compares `pop_data`, `count`, `full` and `empty` with the queue model, then applies the push and pop that the DUT accepts at that edge.

5.1 Structure

Component	Implemented as	Responsibility
Model	`logic [WIDTH-1:0] model_q[$]`	SystemVerilog queue holding the expected FIFO contents, oldest first
Check and model update	One `always @(posedge clk)` block	Compare `pop_data`, `full`, `empty`, `count` with the model; then apply the push/pop the DUT accepts at this edge
Reset	`task automatic apply_reset()`	Idle inputs, `rst_n` low for a few cycles, release with `<=`
Stimulus	`task automatic drive(push, data, pop)`	Drive one cycle of inputs with `<=`, then wait for the next rising edge
Report and watchdog	As in Lab R1	Exactly one `TEST PASSED` / `TEST FAILED`; a hang is a failure

5.2 Required Tests

Test term — Directed and corner-case tests. A directed test is a hand-written sequence with a known expected result aimed at one feature (`fill_drain`: fill, then drain). A corner-case test is a directed test at the edges of the specification — full and empty, push while full, pop while empty, push and pop in the same cycle at full and at empty (`simultaneous`). Most FIFO bugs live exactly there.

`TEST`	Content
`smoke` (default)	Push three values, pop them back with idle cycles in between
`fill_drain`	Fill to full, push while full (must be ignored), drain to empty, pop while empty (must be ignored), then fill again past the pointer wrap point
`simultaneous`	Push and pop in the same cycle at empty (push accepted, pop ignored), at a mid level (`count` unchanged) and at full (both happen)

Data values are deterministic (for example, a counter or a pattern derived from the entry index) so that a wrong entry or a lost entry is visible in the log. An unknown `+TEST=` name is an error. Every test must pass for the three parameter sets. Random stimulus, back-pressure, plusargs such as `NUM_TXN`, regressions and assertions are added in Lab V1.

5.3 Skeleton

```
// hw/ip/sync_fifo/tb/tb_sync_fifo.sv
module tb_sync_fifo #(
  parameter int unsigned WIDTH = 8,
  parameter int unsigned DEPTH = 16
);
  localparam int unsigned TimeoutCycles = 100_000;

  logic                       clk = 1'b0;
  logic                       rst_n;
  logic                       push, pop, full, empty;
  logic [WIDTH-1:0]           push_data, pop_data;
  logic [$clog2(DEPTH+1)-1:0] count;

  logic [WIDTH-1:0] model_q[$];
  string            test_name = "smoke";
  int unsigned      n_checks  = 0, n_errors = 0;

  initial forever #5 clk = ~clk;

  sync_fifo #(.WIDTH(WIDTH), .DEPTH(DEPTH)) u_dut (.*);

  function void check(bit ok, string what);
    n_checks++;
    if (!ok) begin
      n_errors++;
      $error("%s mismatch, model size %0d", what, model_q.size());
    end
  endfunction

  // ---- check, then update the model, at every rising edge ----
  // Runs in the Active region: it sees the values before this edge,
  // the same values the DUT samples. The stimulus uses <=, so its
  // updates of this edge are not visible here yet -> no race.
  always @(posedge clk) begin
    if (rst_n) begin
      bit pop_ok, push_ok;
      check(empty === (model_q.size() == 0),     "empty");
      check(full  === (model_q.size() == DEPTH), "full");
      check(count === model_q.size(),            "count");
      if (model_q.size() > 0)
        check(pop_data === model_q[0], "pop_data");
      pop_ok  = pop  && (model_q.size() > 0);
      push_ok = push && (model_q.size() < DEPTH || pop_ok);
      if (pop_ok)  void'(model_q.pop_front());
      if (push_ok) model_q.push_back(push_data);
    end
  end

  task automatic apply_reset();
    rst_n <= 1'b0;  push <= 1'b0;  pop <= 1'b0;  push_data <= '0;
    repeat (3) @(posedge clk);
    rst_n <= 1'b1;
    @(posedge clk);
  endtask

  // One cycle of stimulus. Called right after a rising edge; the DUT
  // samples these values at the next rising edge.
```

```
  task automatic drive(bit do_push, logic [WIDTH-1:0] data, bit do_pop);
    push      <= do_push;
    push_data <= data;
    pop       <= do_pop;
    @(posedge clk);
  endtask

  task automatic run_fill_drain();
    for (int i = 0; i < DEPTH; i++) drive(1'b1, WIDTH'(i + 1), 1'b0);
    drive(1'b1, '1, 1'b0);              // push while full: ignored
    // TODO: drain to empty, pop while empty, fill again past the wrap
  endtask

  // TODO: run_smoke(), run_simultaneous(), report(), watchdog

  initial begin
    void'($value$plusargs("TEST=%s", test_name));
    $display("CONFIG: WIDTH=%0d DEPTH=%0d TEST=%s", WIDTH, DEPTH, test_name);
    apply_reset();
    case (test_name)
      "fill_drain": run_fill_drain();
      // TODO: "smoke", "simultaneous"
      default: begin n_errors++; $error("unknown TEST=%s", test_name); end
    endcase
    drive(1'b0, '0, 1'b0);              // let the last operation be checked
    report();
    $finish;
  end
endmodule
```

Read the comment on the checker block until you can explain it in terms of the scheduling regions: it is the reason the whole testbench is race-free. The model update reproduces rule 2 and rule 3 of the specification in four lines; if your RTL and your model disagree, decide from the specification which one is wrong.

6 Requirements and Tasks

1. Create the branch `lab/r2-sync-fifo`.
2. Write `doc/spec.md` (interface, parameters, behavior).
3. Implement `rtl/sync_fifo.sv` following all coding rules of Lab R1, with typed parameters, derived `localparam`s and the elaboration-time `DEPTH` check.
4. Write `doc/design.md`: block diagram, the pointer-wrap design note of Section 2.4 with the `DEPTH = 5` table, and why `full` / `empty` are registered.
5. Make `make lint` clean for the three parameter sets. Waivers, if any, go into `lint/sync_fifo.vlt` with a justification.
6. Write `tb/tb_sync_fifo.sv` and `tb/tb.f`: tests `smoke`, `fill_drain` and `simultaneous`, driven with `<=`, checked against a queue model, all passing for the three parameter sets.
7. Show once that `DEPTH=1` fails to build with your error message, and that a deliberately broken pointer wrap is caught by `fill_drain` with `DEPTH=5` but not with `DEPTH=16` (then revert it). Record both in the README.
8. Do the race demonstration of Section 4.6: predictions first, then results, then explanation.

9. Write the README, open the merge request, tag `r2-done` after approval. `make -C $REPO_ROOT check-all` must pass, including `seq_mul` from Lab R1.

`sync_fifo` is synthesized for two parameter sets in Lab P1, and the effect of `WIDTH` and `DEPTH` on its critical path is analyzed in Lab P2. Nothing in this lab requires Yosys or OpenSTA.

7 Repository and Flow

7.1 Files created in this lab

```
hw/ip/sync_fifo/
+-- README.md              block summary, how to run, results, race demo, limitations
+-- Makefile
+-- doc/
|   +-- spec.md            interface, parameters, behavior
|   +-- design.md          block diagram, pointer-wrap note, DEPTH=5 table
+-- rtl/
|   +-- sync_fifo.sv
|   +-- sync_fifo.f
+-- lint/
|   +-- sync_fifo.vlt      waivers (may be empty), each justified
+-- tb/
    +-- tb_sync_fifo.sv    directed smoke testbench (<= driving, queue model)
    +-- tb.f
    +-- race_demo.sv       scheduling exercise, not in any file list
```

7.2 File lists and Makefile

```
// hw/ip/sync_fifo/rtl/sync_fifo.f
+incdir+${REPO_ROOT}/hw/ip/sync_fifo/rtl
${REPO_ROOT}/hw/ip/sync_fifo/rtl/sync_fifo.sv
```

```
// hw/ip/sync_fifo/tb/tb.f
-f ${REPO_ROOT}/hw/ip/sync_fifo/rtl/sync_fifo.f
${REPO_ROOT}/hw/ip/sync_fifo/tb/tb_sync_fifo.sv
```

```
# hw/ip/sync_fifo/Makefile
IP         := sync_fifo
DUT_TOP    := sync_fifo
TB_TOP     ?= tb_sync_fifo
RTL_FLIST  := rtl/sync_fifo.f
# V phase: dv/dv.f, TB_TOP=tb_top
TB_FLIST   ?= tb/tb.f
PARAMS     ?= WIDTH=8 DEPTH=16
TEST       ?= smoke
include $(REPO_ROOT)/flow/mk/common.mk
include $(REPO_ROOT)/flow/mk/lint.mk
include $(REPO_ROOT)/flow/mk/sim_verilator.mk
# include $(REPO_ROOT)/flow/mk/regress.mk   (V1)
# include $(REPO_ROOT)/flow/mk/syn.mk       (P1)
# include $(REPO_ROOT)/flow/mk/sta.mk       (P2)
```

The `SDC` line of the template is added in Lab P2. No shared file in `flow/` changes: if your L0 flow

needs a change to support this IP, it was not generic, and the fix belongs in a separate commit that also keeps `counter` and `seq_mul` passing.

7.3 Commands

```
cd $REPO_ROOT/hw/ip/sync_fifo

make lint                                          # default PARAMS
make lint PARAMS="WIDTH=32 DEPTH=5"
make lint PARAMS="WIDTH=1 DEPTH=2"

make sim                                           # TEST=smoke
make sim TEST=fill_drain
make sim TEST=simultaneous PARAMS="WIDTH=32 DEPTH=5"
make sim TEST=fill_drain PARAMS="WIDTH=1 DEPTH=2" WAVES=1 && make waves

make -C $REPO_ROOT check-all
```

Each parameter set gets its own build directory, so changing `PARAMS` never reuses a model compiled for another configuration. For example, the second set builds into:

`build/sync_fifo/WIDTH-32_DEPTH-5/sim/`

8 Deliverables

The merge request from `lab/r2-sync-fifo` must contain:

- All files of the tree above; nothing generated.
- `doc/spec.md` and `doc/design.md` as described in the tasks, including the pointer-wrap note and the `DEPTH = 5` table.
- `README.md` with: how to run every target used in this lab; a results table (test × parameter set, number of checks, pass/fail); the `DEPTH=1` build error and the broken-wrap experiment; the race demonstration (predictions, results on Verilator and, if available, on an event-driven simulator, and the explanation in terms of scheduling regions); known limitations.

9 Assessment

Criterion	Weight
FIFO functional correctness and RTL quality (coding rules, lint-clean)	30%
Parameterization: any `DEPTH` ≥ 2, derived constants, check, TB passes parameters	15%
Design documentation: pointer-wrap note, `DEPTH = 5` table	10%
Scheduling understanding and race demonstration	25%
Smoke testbench: `<=` driving, queue model, directed tests, three parameter sets	15%
Reproducibility and Git workflow	5%
Total	100%

10 Review Questions

1. At a rising edge, in which region does the DUT read its inputs, and in which region do its outputs change?
2. Your testbench drove with `=` at the falling edge in Lab R1 and drives with `<=` at the rising edge now. Why are both race-free?
3. Why is `#0` not a solution to a race?
4. In the race demonstration, what did `$display` and `$strobe` print for the same variable, and why?
5. Why can't a plusarg change `DEPTH`? What happens if `PARAMS` changes but the build directory is reused?
6. Walk through your pointer wrap for `DEPTH = 5`. Why would `ptr + 1` alone pass every test at `DEPTH = 16`?
7. Why does `count` need `$clog2(DEPTH+1)` bits? What is `$clog2(16)`, `$clog2(17)` and `$clog2(1)`?
8. Why must `full` and `empty` not depend combinationally on `push` and `pop`?
9. What does first-word fall-through mean for a consumer, compared with a FIFO whose data appears one cycle after `pop`?
10. Your checker block reads `push` and the stimulus task writes it at the same edge. Why is that not a race?

11 References

- IEEE Std 1800-2017, *SystemVerilog Language Reference Manual*: Clause 4 (scheduling semantics), Clause 6.20 (parameters), Clause 20.8 (`$clog2`), Clause 20.11 (elaboration system tasks).
- Clifford E. Cummings, Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!, SNUG 2000.
- Clifford E. Cummings, Simulation and Synthesis Techniques for Asynchronous FIFO Design, SNUG 2002 (pointer and full/empty techniques; read it now for the synchronous part, again in Lab T-RTL1).
- Verilator User's Guide: command-line arguments (`-f`, `-G`) and the section on how Verilator orders processes compared with event-driven simulators.
- *SystemVerilog for Design*, Sutherland, Davidmann and Flake: chapters on procedural blocks and parameterized modules.
- lowRISC, Verilog/SystemVerilog Coding Style Guide (parameter naming and FIFO examples in OpenTitan's `prim_fifo_sync`).